

KVMillésime - Mise en fût

Comme *KVMillésime - Vendanges tardives*, ce challenge est orienté ingénierie de la crypto. Il montre comment, même en tant que guest d'un système virtualisé, on peut faire leaker des secrets du kernel *host*.

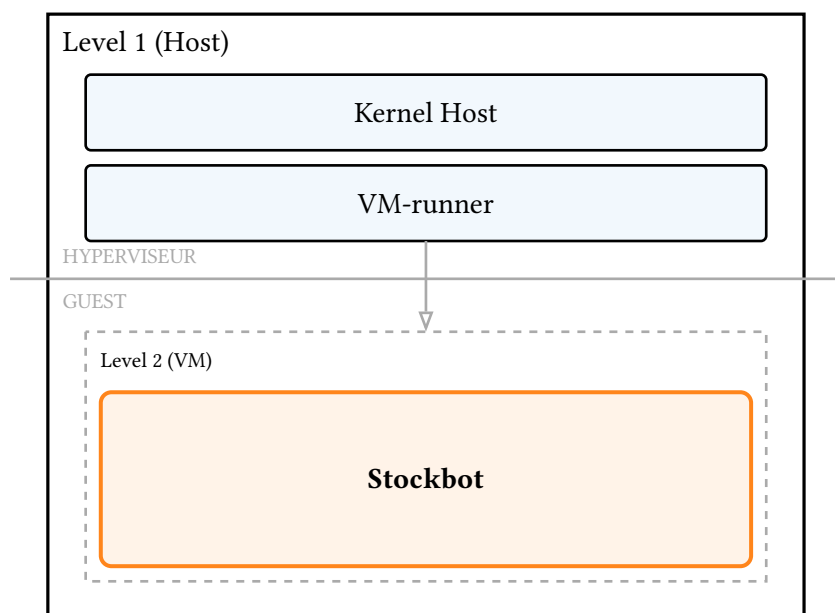
Gist

La vérification "crypto" est déportée de la VM guest au kernel de l'hôte. Ce kernel renvoie le nombre de retenues opérées lors d'une manipulation arithmétique avec le secret à faire fuiter. Cela suffit à bruteforcer le secret.

Note: dans la vraie vie, le kernel ne renverrait pas directement le nombre de retenues. Mais faire une opération avec retenue prends légèrement plus de temps que sans. Avec un peu d'adresse et beaucoup de bruteforce, on arriverait au même résultat que le compteur ci-présent.

Tour d'horizon

La base de code est plus conséquente. Ci-contre une représentation du setup :



Le socat auquel le challenger a accès expose une machine L1. Cette machine présente

- un module kernel customisé pour implémenter un vmcall spécifique; le diff avec un kernel standard est donné dans `diff_vmx.c` ;
- un runner de VM minimal ;

Le runner de VM instancie une VM L2, le *Stockbot*. Comme il s'agit d'une VM minimale, le code ne peut pas inclure la plupart de la libc : tout est réimplémenté à la main, expliquant les `io.h` ou `smallstring.h`.

Dans `L2_stockbot/main.c`, on voit la liste des commandes disponibles, dont la fonction `show_flag()` appelée pour `admin_token`. Pour l'appeler, il faut valider `is_licensed()`, et donc `validate_license()`.

Son code se trouve dans `license.c` :

```
int validate_license(uint8_t *p_license_key) {
    int ret;
    // ...
}
```

```

asm volatile (
    "vmcall"
    : "=a"(ret), "=d"(diagnostic_metric)
    : "a"(LICENSE_MAGIC), "c"(p_license_key)
    : "memory"
);

// ...
char log_msg[70] = "TRADING_LATENCY_PROFILE: Inserted hardware wait state: ";
char metric_str[20] = {0};
my_itoa(diagnostic_metric, metric_str);
int cur = my_strlen(log_msg);
my_memcpy(log_msg + cur, metric_str, my_strlen(metric_str) + 1);
log_append(timestamp, log_msg);

return ret;
}

```

Ainsi, il fait un `vmcall` avec un magic dans `rax`. Ce `vmcall` va être pris en charge par le kernel, `L1_kernel/vmx.c`. Le reste de la fonction journalise la valeur renvoyée par le kernel.

Dans `L1_kernel/vmx.c`, on peut voir que (i) le kernel a un handler custom pour `vmcall` quand `rax = 0x1337` – c’est le cas ici –, et (ii) qu’il répond à l’appelant *via* `rax` (résultat de `constant_time_compare_128`) et `rcx` (valeur de `wait_state_telemetry`).

Enfin, `constant_time_compare_128` vérifie que la clef 128 bits donnée par l’utilisateur correspond au secret généré au boot, en soustrayant `user_value - kernel_secret` ; il renvoie également le nombre de retenues opérées pendant la soustraction *arrondies au multiple de 5 inférieur*.

L’objectif est donc :

1. Dédurre ou bruteforcer le secret du kernel par l’appel de la fonction `license` ;
2. Utiliser ce secret pour s’authentifier dans Stockbot ;
3. Appeler la commande `admin_token` pour obtenir le flag.

Pour ce faire, on

- contrôle la valeur contre laquelle est comparée la clef ;
- connaît, pour une entrée donnée, le nombre de retenues nécessaires à la soustraction du secret à notre valeur, au travers des logs de Stockbot.

Attaque

Résolution en supposant que le kernel renvoie le nombre exact de retenue

On cherche à attaquer la clef $K = k_{\{127\}} \dots k_0$, avec l’input $U = u_{\{127\}} \dots u_{\{0\}}$. Soit $(r_j)_j \in \{0, 1\}$ la valeur de retenue au tour j de la boucle de soustraction.

Première observation Mettre les bits bas de l’input à 1 forge un “bouclier à retenue”.

L’opération de soustraction commence par les bits bas. La retenue n’est levée que si $u_j - k_j - r_j < 0$. On montre facilement par récurrence que $\forall i < j, r_i = 0$.

Deuxième observation On ne peut pas deviner le bit le plus haut. Il n’entre pas en jeu dans le calcul de retenue.

Troisième observation En connaissant les m bits hauts (hors le bit 127), on peut déduire la valeur du bit $u_{\{127-m-2\}}$ grâce au nombre de retenues.

On fixe $\forall j < 127 - m - 2, u_j = 1$ pour faire un bouclier à retenue.

On note $\tilde{U} = u_{\{127\}} \dots u_{\{127-m-1\}} 11 \dots 1$ et $U = u_{\{127\}} \dots u_{\{127-m-1\}} 01 \dots 1$ les deux guess possibles pour la valeur du bit $127 - m - 2$.

On note $\tilde{R}$ le nombre de retenues annoncées par le serveur à l'envoi de $\tilde{U}$, et R pour U .

Deux cas possibles :

- Si $k_{\{127-m-2\}} = 1$, alors $\tilde{R} = R = 0$;
- Si $k_{\{127-m-2\}} = 0$, alors $\tilde{R} = 0$ et $R = m$.

En effet, pour ce dernier cas, le bit $127 - m - 2$ génère une retenue. Et comme tous les bits plus haut sont égaux (hypothèse), la retenue est propagée jusqu'au bit de poids fort.

Ainsi, par récurrence, on peut bruteforcer tous les bits sauf celui de poids fort. On n'aura qu'à tester les deux clefs possibles.

Résolution lorsque le kernel tronque au multiple de 5 le plus proche

On va décliner la stratégie précédente.

Fonctionnellement, ne donner que par multiple de cinq ne pose de problèmes que pour les bits haut. On se convainc facilement que, si les tests nous mettent actuellement dans le "paquets des R à 10", choisir le mauvais bit nous fais tomber dans le paquet précédent (des R à 5, pour cet exemple). La troncation n'est donc un problème que lorsqu'on est déjà au paquet le plus bas, *ie.* les 5 bits haut.

Pour résoudre le problème, on teste simplement toutes les valeurs de bit haut en parallèle. On aura 32 valeurs possibles de clefs que l'on testera successivement en demandant les droits admin au stockbot.